

Can Deep Neural Networks Improve Compression of Very Large Scientific Data?

Muhammad Alhumaidi

King Abdullah University of Science and Technology
Thuwal, Saudi Arabia
muhammad.humaidi@kaust.edu.sa

Spiros Skiadopoulos

University of the Peloponnese
Tripoli, Greece
spiros@uop.gr

Guozhong Li

King Abdullah University of Science and Technology
Thuwal, Saudi Arabia
guozhong.li@kaust.edu.sa

Panos Kalnis

King Abdullah University of Science and Technology
Thuwal, Saudi Arabia
panos.kalnis@kaust.edu.sa

Abstract

Error-bounded lossy compression is a fundamental technique for managing the rapidly growing volumes of scientific data produced by modern simulations and observational instruments. Most state-of-the-art compressors follow a prediction-residual paradigm, where compression effectiveness depends on the quality of the predictor: more accurate predictions generate smaller residuals that are easier to compress. This observation raises a question: can modern machine learning models serve as superior predictors for scientific data compression? Answering this question directly is challenging because developing compression-specific ML predictors requires substantial resources. Instead, we leverage the climate domain where highly accurate pretrained weather forecasting foundation models already exist, making them an ideal testbed. We present a framework that integrates spatial and temporal deep learning models into a conventional error-bounded compression pipeline. The framework supports auto-regressive forecasting models and avoids error accumulation. Using ERA5 climate data as a representative large-scale scientific dataset, we evaluate three distinct ML predictors: a VAEformer-based codec (CRA5), a graph neural network forecaster (GraphCast), and a vision-transformer forecaster (Aurora), against the state-of-the-art compressor SZ3.1 under identical quantization and entropy-coding backends. Our evaluation over approximately 1.7 TB of data reveals a surprising result: although ML predictors generate more accurate predictions and can improve reconstruction quality by up to 91% while achieving up to 9.6 $\times$ higher compression ratios for highly predictable variables, they do not improve overall dataset-level compression ratio. We show that prediction accuracy alone is insufficient: the spatial structure of the resulting residuals plays a decisive role in entropy coding efficiency. Our findings provide new insight into the relationship between prediction and compression, establishing the opportunities and limitations of using foundation models as predictors for scientific data compression.

1 Introduction

Modern sensing and simulation generate scientific datasets at the terabyte to petabyte scale across many domains, including weather and climate modeling [7, 18, 22], large-scale reanalysis [14], cosmological simulations [33], computational fluid dynamics and turbulence [1, 27], combustion [6], and seismic imaging [19]. At these volumes, storage, network, and compute infrastructures become

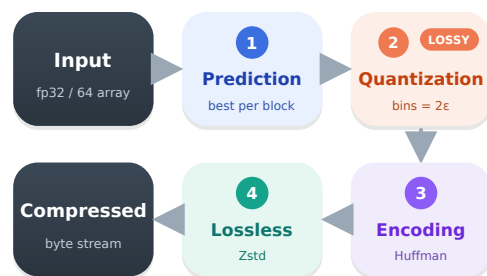


Figure 1: Typical (e.g., SZ3.1 [41]) error-bounded lossy compression pipeline. We focus on the Prediction module.

the dominant bottleneck [9], making data *compression* essential for tractable storage, transfer, and analysis.

Lossless compression, as implemented in scientific data libraries (e.g., NetCDF [37], HDF5¹) through algorithms like Zlib² and Zstandard³, recovers data exactly but compression ratios rarely exceed a factor of 2 [42]. In contrast, *lossy* compression [15, 17, 20] can push compression ratios to thousands, but without any control over the errors the decompressed data may be unsuitable for scientific use. This led the scientific community toward a middle ground: *error-bounded lossy* compression. The idea is to allow some loss of precision, but to guarantee that a user-chosen error metric, such as relative point-wise error, to never exceed a predefined bound ϵ .

Many error-bounded lossy compression methods are built around the *prediction-residual* architecture depicted in Figure 1. A prediction module approximates input data⁴ points; the error-bounding module then quantizes and encodes only the residual between that prediction and the true value. Existing compressors approach the prediction problem in several ways: HPEZ [32] and SZ3.1 [41] combine multiple interpolation schemes to predict from spatially neighboring points, while SPERR [25] and ZFP [29] operate in a wavelet-transformed domain. Recently, deep neural networks have

¹<http://www.hdfgroup.org/HDF5>

²<http://www.zlib.net/>

³<https://github.com/facebook/zstd/>

⁴Scientific data encompasses various types, including numerical, categorical, text, and image data; this paper focuses on *numerical* data.

entered the picture, giving rise to autoencoder-based SZ [31], coordinate network-based compressors [10], and super-resolution network-based SZ [30].

The pipeline of Figure 1 carries a consequential implication: the quality of the predictor determines the compression ratio. A good predictor keeps residuals small and tightly clustered, producing a data stream that entropy coders can compress aggressively.

This observation raises the central question of our work: *if the simple, general-purpose predictor at the heart of these compressors were replaced by a highly elaborate, ML-trained model, would it serve as a more effective prediction front-end and yield a better compression pipeline?* Given the centrality of predictor quality to compression ratio, a learned model that closely tracks the data, i.e., one able to exploit physical priors and long-range spatiotemporal dependencies that are invisible to polynomial or wavelet-based predictors, should in principle produce residuals of substantially lower variance, directly translating to higher compressibility.

However, training and validating such a model from scratch is a significant task. It would require large amounts of training data, expensive computational budgets, careful architectural design, and extensive empirical validation. Such resources would need to be justified before the viability of the compression use case is even established.

A pragmatic and scientifically sound alternative is to focus on a domain where highly accurate ML models *already exist* and to repurpose these *pretrained* models. Climate is exactly such a domain. Recent advances in data-driven weather prediction have produced models such as GraphCast [21], Aurora [4], Pangu-Weather [3], and FourCastNet [34]. The models are trained on the massive ERA5 [14] global reanalysis dataset produced by the European Centre for Medium-Range Weather Forecasts (ECMWF), which provides hourly estimates of a wide range of atmospheric, land-surface, and oceanic variables at a spatial resolution of 0.25° from 1940 to the present. The ML models already implicitly encode the spatiotemporal structure of ERA5 in their learned representations. Using them as predictors within a prediction-residual compression framework is therefore not merely a shortcut but a principled proxy experiment: their performance on the compression task will give us a reliable signal about whether deep learning-based prediction is a viable direction for scientific data compression, and will expose the bottlenecks that any such approach, including purpose-trained models, would need to overcome.

In this paper, we present a systematic evaluation of the prediction-residual compression framework applied to ERA5 data, comparing three qualitatively different deep learning-based prediction strategies against an established baseline, within a shared error-bounded back-end. (i) CRA5 [12] is a *learned spatial* predictor based on a VAEformer encoder-decoder that reconstructs each variable independently, without conditioning on adjacent timesteps. It was specifically trained to compress ERA5 data without error bound. It can be integrated in the error-bounded pipeline of Figure 1 in a straight-forward manner, by replacing the Prediction module. (ii) GraphCast [21] is a mesh-based graph neural network, that serves as *temporal* predictor by receiving the current state as input and predicting the next few states. To integrate GraphCast in the error-bounded compression framework, we need to modify the pipeline in an *auto-regressive* way: we route each prediction through

the error-bounding module, where the residual is calculated. The residual is combined with the prediction to generate a “corrected” current state that is used as the next input for the predictor; refer to Section 3.2 for details. (iii) Aurora [4] is a patch-based vision transformer that also serves as temporal predictor. Similar to GraphCast, in order to ensure error-bounded compression, we integrate Aurora within our auto-regressive compression pipeline. (iv) SZ3.1 [41] serves as the classical *spatial polynomial* baseline. All four methods share an identical quantization and entropy coding back-end, for fair comparison of each prediction strategy.

Our evaluation spans 1,997 autoregressive steps (~ 500 days) across 9 atmospheric variables at three relative error bounds $\epsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. Our results reveal that, when considering the dataset as a whole, none of the ML-based methods can match the compression ratio of SZ3.1, because their residuals, while smaller in magnitude, lack the spatial coherence that entropy coding exploits. Interestingly, if we consider each variable separately, for some highly-predictable variables the ML-based methods achieve up to $9.6\times$ higher compression ratios than SZ3.1. Note that, even though the point-wise error is bounded for all methods, the fidelity of the decompressed results is different: SZ3.1 generally results in larger average error, and the error distribution tends to generate structured artifacts that are undesirable for scientific analysis. In contrast the mean absolute error (MAE) of CRA5 and Aurora is up to 91% and 34% lower than SZ3.1, respectively, while the error distribution is much smoother. Interestingly, the auto-regressive design for Aurora and GraphCast is stable over nearly 2,000 steps with no error accumulation. In summary, the answer to our main question is nuanced: when used as predictors in an error-bounded lossy compression pipeline, these elaborate ML models can dramatically improve reconstruction fidelity, yet do not by themselves yield higher compression ratios.

Our contributions are as follows:

- We develop an auto-regressive framework that enables temporal forecasting models to be used as predictors in an error-bounded lossy compression pipeline, without error accumulation.
- We integrate three representative elaborate ML models, a VAEformer (CRA5), a mesh-based GNN (GraphCast) and a patch-based vision transformer (Aurora), within an identical error-bounded quantization and entropy coding back-end, enabling a fair comparison of prediction strategies for ERA5 compression.
- We present a comprehensive large-scale (around 1.7TB of data) experimental evaluation with analysis per-variable, per-level, and per-pixel. We show that ML-based predictors can significantly improve reconstruction fidelity and may even improve compression ratio for some variables; however, they cannot match the compression ratio of the simpler predictors of SZ3.1 for the entire ERA5 dataset.

2 Background

2.1 Error-bounded lossy compression

A temporal scientific dataset $\mathcal{X}$ captures data across multiple timesteps, organized in a set of *variables* such as temperature, humidity,

etc. Each variable $X \in \mathcal{X}$ is a multi-dimensional matrix of real numbers. For the rest of this paper we assume that $X \in \mathbb{R}^{T \times M \times N}$, where T is the number of timesteps and M, N are the number of rows and columns, respectively.

For a variable X , let $|X|$ be the size of the original data in bytes. Consider some compression method, and let $|Z|$ be the total size of the compressed data in bytes; this includes all the *overhead* data needed for decompression, such as metadata, initialization values for auto-regressive predictors, learned latent representations, ML model weights, etc. The compression ratio ρ is defined as:

$$\rho = \frac{|X|}{|Z|} \quad (1)$$

For the entire dataset $\mathcal{X}$, the formula is generalized to $\rho = \frac{|\mathcal{X}|}{|\mathcal{Z}|}$. In case part of the overhead (e.g., the ML model weights) is considered common knowledge, for fairness we will exclude that part from the calculation of $|Z|$ and denote compression ratio as $\bar{\rho}$.

Error-bounded lossy compression. For a variable X , let ϵ be a user-specified *relative* error bound. Let C be a compression function that generates the compressed version Z of X , i.e., $Z = C(X)$, and let D be a decompression function that generates the decompressed version X' of Z , i.e., $X' = D(Z)$. The process is lossy, therefore X' is an approximation of X . The goal is to define a pair of functions $\langle C, D \rangle$ such that the compression ratio ρ of X is maximized, while the relative difference between each original data point X_i and its decompressed counterpart $X'_i = D(C(X))_i$ does not exceed the error bound ϵ . Formally:

$$\arg \max_{\langle C, D \rangle} \frac{|X|}{|C(X)|}, \quad \text{such that} \quad \frac{|X_i - D(C(X))_i|}{r_X} \leq \epsilon \quad (2)$$

where r_X returns the range of values in the variable.

2.2 Prediction-residual compression pipeline

Many error-bounded lossy compression methods [10, 25, 29–32, 41] follow a pipeline similar to that of Figure 1, their main differences being the way they prepare the input data (i.e., step-0) and their prediction method (i.e., step-1). For the rest of this paper we will focus on SZ3.1 [41], which is a popular state-of-the-art general-purpose method. We will use SZ3.1 as a representative high-performance baseline, with the caveat that, for specific application domains, some other method with a particular optimization heuristic may perform slightly better.

SZ3.1 uses a *spatial* predictor for step-1 of Figure 1: it fits a polynomial interpolant (see Figure 2) to neighboring points within the same timestep k to predict values $p_{X,k}$ of variable X , computes the residual $d_{X,k}$ against the true values $t_{X,k}$, quantizes it under error bound ϵ , and entropy-codes the result. SZ3.1 has no access to temporal information; each timestep is compressed independently. The residual is computed as:

$$d_{X,k} = t_{X,k} - p_{X,k} \quad (3)$$

Error-bounded quantization (step-2). To enforce a relative point-wise error bound ϵ , the quantization bin width is defined as a function of ϵ and the global value range of each variable. Due to quantization, this is the *only lossy step in the pipeline*. For a variable X , the global minimum and maximum are computed in advance



Figure 2: Polynomial prediction example. Black and white circles represent processed and unprocessed data points, respectively; arrows denote interpolation.

across all timesteps $1 \leq k \leq T$ and spatial locations $1 \leq i \leq M$, $1 \leq j \leq N$; then the range is:

$$r_X = \max_{k,i,j} t_{X,k}^{(i,j)} - \min_{k,i,j} t_{X,k}^{(i,j)} \quad (4)$$

The quantization bin width is set to:

$$\delta_X = \epsilon \cdot r_X \quad (5)$$

This bin width is fixed for the entire compression sequence; it does not vary across timesteps. Each residual value is then mapped to an integer bin index:

$$q_{X,k}^{(i,j)} = \left\lfloor \frac{d_{X,k}^{(i,j)}}{\delta_X} + \frac{1}{2} \right\rfloor, \quad \text{where} \quad q_{X,k} \in \mathbb{Z}^{M \times N} \quad (6)$$

Entropy coding (step-3) and compression (step-4). Huffman encoding [16] compresses data *losslessly* by assigning shorter binary codes to frequently occurring characters and longer codes to rare ones. In step-3 the SZ3.1 pipeline uses the Zlib⁵ implementation of Huffman encoding to compress the integer quantized residuals $q_{X,k}$. Assuming that $p_{X,k}$ is an accurate prediction of $t_{X,k}$, the residuals $d_{X,k}$ are small in magnitude, and the corresponding bin indices $q_{X,k}$ cluster around zero. This produces a highly skewed integer distribution that Huffman coding can exploit aggressively.

The above mentioned quantization pipeline excludes outliers which are marked as “unpredictable”. The true values of those outliers are combined with the output of the Huffman encoder and are further compressed *losslessly* in step-4 by the Zstd⁶ algorithm.

2.3 Decompression

Decompression follows the inverse pipeline of Figure 1. The crucial step is dequantization, which recovers an approximation of the residual:

$$\tilde{d}_{X,k}^{(i,j)} = q_{X,k}^{(i,j)} \cdot \delta_X \quad (7)$$

The error-corrected state is then:

$$p'_{X,k} = p_{X,k} + \tilde{d}_{X,k} \quad (8)$$

which satisfies the relative error bound by construction:

$$\left| p'_{X,k}^{(i,j)} - t_{X,k}^{(i,j)} \right| \leq \frac{\delta_X}{2} = \frac{\epsilon \cdot r_X}{2}. \quad (9)$$

⁵<http://www.zlib.net/>

⁶<https://github.com/facebook/zstd/>

3 Methodology

Section 2 established that, in the prediction-residual pipeline, the predictor is the component that determines the compression ratio: predictions that are accurate produce residuals that are small in magnitude and tightly clustered around zero, leaving the entropy coder little to encode. The quality of the predictor is therefore crucial for error-bounded lossy compression of scientific data. Classical compressors such as SZ3.1 rely on simple, general-purpose predictors (e.g., spatial polynomial interpolation) that are cheap to evaluate but blind to the physical priors and long-range spatiotemporal dependencies that govern scientific fields. Our work aims to investigate whether an elaborate, ML-trained model can act as a more accurate predictor to improve the compression pipeline.

Answering this question for scientific data *in general* is not practical. A predictor purpose-built for compression would require an excessive amount of resources that are beyond the reach of most academic environments. Rather than train such a model from scratch, we adopt a pragmatic and scientifically sound alternative: we restrict attention to a single domain in which highly accurate ML models *already exist* and re-purpose them as predictors behind a fixed error-bounded back-end. Below we argue that climate is an appropriate target for such an investigation for two reasons: there exists a plethora of highly-curated open source data, and there exist several accurate pre-trained ML models adopted by the scientific community.

ERA5 dataset. ERA5 [14] is a global climate reanalysis dataset that combines past weather observations with advanced numerical models to create a continuous record of Earth’s atmosphere, land, and oceans. It was developed by the European Centre for Medium-Range Weather Forecasts (ECMWF). ERA5 unifies data from satellites, weather balloons, and ground sensors to provide a highly detailed, historically consistent map of global climate conditions dating back to 1940. ERA5 contains hourly values of a variety of variables such as temperature, humidity and wind at several pressure levels (i.e., altitudes) in the atmosphere, on a 0.25° latitude-longitude global grid that corresponds to a 721×1440 matrix; an example is shown in Figure 3. A variable at a specific pressure level is a 3D matrix $X \in \mathbb{R}^{T \times M \times N}$ (refer to Section 2). The total size of ERA5 exceeds 5 PB, depending on the format; in this study we use an extract of approximately 1.7 TB.

ML-based predictors. There exist several pre-trained ML models [3, 4, 12, 21, 34] that forecast accurately the atmospheric conditions. In this study, we focus on three representative such models: CRA5 [12] is a learned spatial codec (i.e., VAEformer) built specifically to compress ERA5. It is the most direct learned competitor to a classical compressor, so it is the natural learned-spatial point of comparison. GraphCast [21], a graph neural network, and Aurora [4], a vision transformer, are temporal forecasters. We select them for three reasons: (i) both are among the best forecasting models at the time of this work; (ii) both release documented inference code and pretrained weights, so the evaluation is reproducible; and (iii) they are trained on ERA5, so they already encode its spatial structure and temporal dynamics. We implement all methods using the same quantization and entropy-coding back-end, for fair comparison.

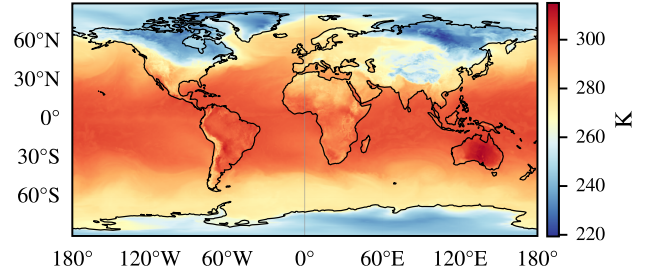


Figure 3: A single ERA5 field: 2-meter temperature (in K) at one timestep, on the 0.25° (721×1440) latitude-longitude grid. The field has clear large-scale spatial structure, warm in the tropics and colder toward the poles.

3.1 CRA5: Learned Spatial Reconstruction

CRA5 [12] is a neural codec built specifically to compress ERA5. Its design follows learned image compression: a transformer-based variational autoencoder (VAE) called the VAEformer, built as a dual-VAE. A main VAE maps each atmospheric state to a quantized latent representation, and a second, hyperprior VAE models the latent distribution and produces the parameters of a Gaussian entropy model for arithmetic coding. The result is a reconstruction of each field on its own. CRA5 compresses an extract of the ERA5 archive (about 226 TB) to roughly 0.7 TB, a ratio around 322 $\times$, but it is rate-distortion optimized, without any point-wise error guarantee.

Fitting CRA5 into the pipeline. CRA5 replaces the Prediction module (i.e., step-1) in the pipeline of Figure 1. We use CRA5 in inference mode with the released weights and no change to the architecture. We employ the CRA5 codec to decode each variable X and use that reconstruction as the prediction $p_{X,k}$ for step k . The residual $d_{X,k} = t_{X,k} - p_{X,k}$ then passes through the same error-bounded back-end of Section 2: the residual is quantized such that the reconstruction is bounded by relative error ϵ , and then the quantized data are compressed by Huffman followed by Zstd. This process transforms CRA5 from an unbounded codec into an error-bounded one.

Decompression is straight-forward, too: the decoder forms the CRA5 reconstruction and applies the error correction of Section 2 to recover $p'_{X,k}$.

3.2 GraphCast: Auto-regressive Prediction

GraphCast [21] is a graph neural network with an encode-process-decode design on a multi-mesh icosahedral grid. It maps the latitude-longitude ERA5 grid onto the mesh, runs 16 learned message-passing layers that move information across spatial scales (the finest mesh has 40,962 nodes), and maps the result back to the grid. The model has 36.7 M parameters and was trained on ERA5 over the period 1979 to 2017. It predicts 5 surface and 6 atmospheric variables across 37 pressure levels. It takes two consecutive states at t_{k-2} and t_{k-1} plus static forcing (solar radiation, land-sea mask) and outputs the state at t_k on the 721×1440 grid at a 6-hour interval. It forecasts more accurately than ECMWF’s operational high resolution forecast system on about 90% of the reported targets.

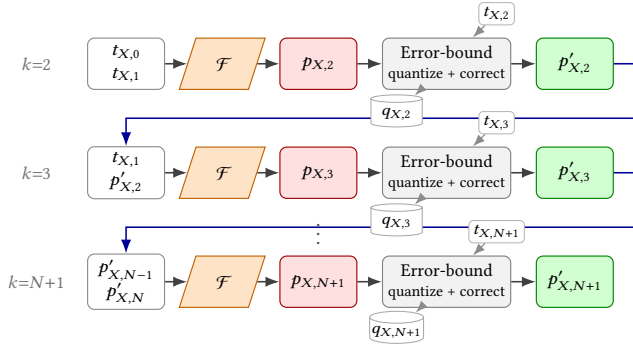


Figure 4: Autoregressive loop for the temporal forecasters (GraphCast, Aurora). Two seed states are stored losslessly. At each step k , $\mathcal{F}$ predicts $p_{X,k}$ from the two preceding corrected states; the error-bounding module stores the quantized residual $q_{X,k}$ and feeds back the corrected state $p'_{X,k}$, keeping every input within $\epsilon r_X/2$ of the truth and preventing drift.

Auto-regressive integration. GraphCast cannot be integrated directly into the pipeline of Figure 1. It is a temporal forecaster, so it predicts the next state from the two preceding ones, but at decode time those preceding states are not available as ground truth. We therefore propose to wrap it in an auto-regressive closed loop and feed back the decompressed (i.e., “corrected”) state.

Let t_k , p_k , and p'_k be the full multi-variable states at step k , that is, the stacks of the per-variable slices $t_{X,k}$, $p_{X,k}$, and $p'_{X,k}$ over all $X \in \mathcal{X}$. We store the first two states t_0 and t_1 losslessly to seed the loop. For each step $k \geq 2$ the model $\mathcal{F}$ predicts from the two preceding *corrected* states:

$$p_k = \mathcal{F}(p'_{k-2}, p'_{k-1}). \quad (10)$$

For each variable X , the residual $t_{X,k} - p_{X,k}$ passes through the same error-bounding back-end (Section 3.1) to generate the stored residual $q_{X,k}$ and the corrected slice $p'_{X,k}$. We assemble the corrected slices into p'_k and feed it forward. Figure 4 shows the loop and Algorithm 1 lists the steps. The compressed output consists of the two seed states and the residual stream $\{q_{X,k}\}$. We use GraphCast in inference mode with the released weights and no fine-tuning.

Decompression. Decompression runs the same pipeline backwards. The decoder loads the seeds t_0 and t_1 , then at each step runs $p_k = \mathcal{F}(p'_{k-2}, p'_{k-1})$, decodes the residual $q_{X,k}$, and applies the same error correction as in compression (Section 2) to recover $p'_{X,k}$. The decoder re-runs $\mathcal{F}$ to reproduce each prediction, so the encoder and decoder must compute the same values; any difference would change the corrected state and violate the bound. This holds because $\mathcal{F}$ runs deterministically and both sides use the same corrected states as input. GraphCast’s default GPU implementation is not bit-reproducible, so we run it in a deterministic configuration on CPUs.

Overhead and the compression ratio. The compressed size $|Z|$ of an auto-regressive method consists of the two seed states plus the residual stream. To compute compression ratio ρ we also add the overhead of the model weights. However, in many practical cases

Algorithm 1: Autoregressive error-bounded compression (GraphCast, Aurora).

Input: Full states $t_0, \dots, t_{N+1}$ of dataset $\mathcal{X}$;
relative error bound ϵ ;
number of prediction steps N ;
pretrained forecasting model $\mathcal{F}$ (GraphCast or Aurora)

Output: Compressed bitstream Z

// Store the two seed states losslessly

```

1  $Z \leftarrow \text{store}(t_0, t_1)$ 
2  $p'_0 \leftarrow t_0; p'_1 \leftarrow t_1$ 
3 for  $k \leftarrow 2$  to  $N + 1$  do
    // Predict all variables jointly from the two
    // corrected states
4    $p_k \leftarrow \mathcal{F}(p'_{k-2}, p'_{k-1})$ 
5   foreach variable  $X \in \mathcal{X}$  do
6      $d_{X,k} \leftarrow t_{X,k} - p_{X,k}$ 
7      $\delta_X \leftarrow \epsilon \cdot r_X$ 
8      $q_{X,k} \leftarrow \lfloor d_{X,k} / \delta_X + 1/2 \rfloor$ 
9      $\Xi \leftarrow \text{encode}(q_{X,k})$ 
    // Append the compressed residual to the
    // stream
10     $Z.\text{append}(\Xi)$ 
11     $p'_{X,k} \leftarrow p_{X,k} + q_{X,k} \cdot \delta_X$ 
    // Assemble the corrected state and slide the
    // window
12     $p'_k \leftarrow \{p'_{X,k} : X \in \mathcal{X}\}$ 
13 return  $Z$ 
```

the model weights are considered a fixed asset shared across many compression tasks. In such a case we exclude that overhead (see Section 2) and calculate compression ratio $\bar{\rho}$. For completeness, in our experimental evaluation we report both quantities.

Stability by construction. Long auto-regressive sequences may drift because small errors at each fed-back state can accumulate. Our approach prevents this. The bound of Section 2 keeps every corrected state within $\epsilon r_X/2$ of the truth at each step, so the state fed to the model is always close to the true value. The correction is applied at every step before the state moves forward, which re-anchors the sequence to the data, avoiding error accumulation. Section 4.5.4 confirms this over the full 1,997-step rollout.

3.3 Aurora: Auto-regressive Transformer

Aurora [4] is also a temporal forecaster that we integrate into the same auto-regressive loop as GraphCast (Section 3.2), with the same seed states, back-end, and feedback of the corrected state p'_k . Consequently, the compression pipeline is the one of Figure 4.

It differs from GraphCast in scale, architecture, and training. Aurora has 1.3 B parameters, far more than GraphCast’s 36.7 M. Its backbone is a 3D Swin Transformer U-Net with a perceiver-based encoder and decoder that map a variable set of input fields to a fixed latent representation (see Figure 5). It splits the global state into patches, takes the two preceding states as input, and processes

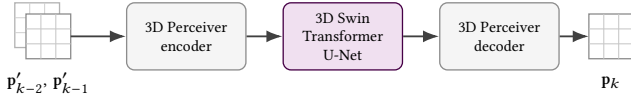


Figure 5: Aurora forecast path. A 3D Perceiver encoder maps the two preceding states p'_{k-2}, p'_{k-1} to a fixed latent, a 3D Swin Transformer U-Net advances it, and a 3D Perceiver decoder produces the next state p_k .

them with an attention mechanism. Unlike GraphCast, which is trained on ERA5 alone, Aurora is pretrained on more data from six sources (including ERA5), and then fine-tuned. It predicts 4 surface and 5 atmospheric variables on 13 pressure levels (50 to 1000 hPa), a different native level set from GraphCast’s 37. These differences change prediction quality, and therefore compressibility.

4 Experimental Evaluation

This section evaluates experimentally whether using an elaborate ML model as predictor can improve error-bounded compression. We report the primary metric, dataset-level compression ratio (Section 4.2), the compression and decompression throughput (Section 4.3), as well as the reconstruction fidelity of each method (Section 4.4). We then perform a detailed ablation study that includes per-variable compression, initialization overhead and auto-regression stability (Section 4.5). Finally, we study the apparent paradox that more accurate predictions need not compress better, through an analysis of residual structure (Section 4.6).

4.1 Experimental Setup

4.1.1 Dataset. Our evaluation uses the ERA5 reanalysis dataset [14], which provides hourly global weather and climate data on a 0.25° grid of the globe. The grid corresponds to a 2D latitude-longitude matrix with $M = 721$ rows and $N = 1440$ columns. As summarized in Table 1, ERA5 contains variables that fall into two categories: (i) single-level, such as surface pressure (sp), are 3-dimensional matrices in $\mathbb{R}^{T \times M \times N}$, where T is the number of timesteps; and (ii) multiple-level variables such as specific humidity (Q), that add an extra dimension for the atmospheric 37 pressure levels (i.e., altitude). In this work, the slice of a multi-level variable at each pressure level is considered a distinct 3D variable and denoted together with its pressure level (e.g., $Q@500\text{hPa}$).

Each variable $X \in \mathbb{R}^{T \times M \times N}$ is compressed independently, with no cross-variable or cross-level information shared during quantization or entropy coding.

We extract $T = 1,997$ consecutive timesteps from the ERA5 dataset at 6-hour resolution, as GraphCast and Aurora operate at this resolution, covering approximately 500 days. Each timestep contains around 4.15 MB of single-precision floating-point values; the total size of the data is around 1.7 TB. Not every variable is predicted by every model; Table 1 reports the per-model coverage. We choose 9 variables common to all methods: (i) single-level 2m temperature (T2m), 10m u - and v -wind (U10, V10), and mean sea level pressure (MSL); and (ii) multi-level temperature (T), geopotential (Z), u - and v -wind (U, V), and specific humidity (Q). Following

Table 1: ERA5 variables used for evaluation, with short names and which models predict each (✓): GraphCast (GC), Aurora (AR), CRA5 (CR); MLevel denotes multiple levels.

Variable	SName	MLevel	GC	AR	CR
Geopotential	Z	✓	✓	✓	✓
Specific humidity	Q	✓	✓	✓	✓
Relative humidity	R	✓			✓
Temperature	T	✓	✓	✓	✓
U component of wind	U	✓	✓	✓	✓
V component of wind	V	✓	✓	✓	✓
Vertical velocity	W	✓	✓		✓
2m temperature	T2m		✓	✓	✓
10m u wind component	U10		✓	✓	✓
10m v wind component	V10		✓	✓	✓
100m u wind component	U100				✓
100m v wind component	V100				✓
Mean sea level pressure	MSL		✓	✓	✓
Total precipitation	TP		✓		✓
Total cloud cover	TCC				✓
Surface pressure	SP				✓

standard weather forecasting practice, we include representative results at the 500 and 850 hPa pressure levels.

4.1.2 Hardware. CRA5 and Aurora were run on an NVidia A100-SXM4-80GB GPU with 80GB HBM, installed on a machine with an AMD EPYC 7713P 64-Core CPU @ 2.0GHz and 512GB RAM with Rocky Linux release 9.4. SZ3.1 and GraphCast were run on an Intel Xeon Gold 6246 12-Core CPU @ 3.30GHz, 3TB RAM machine with Rocky Linux release 9.4. GraphCast was executed on CPU by necessity: its JAX implementation is non-deterministic on GPUs, which is incompatible with the requirements of compression; forcing deterministic GPU execution is no faster than CPU.

4.2 Compression Ratio of the Entire Dataset

We use three relative error bounds $\epsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ to compress the entire dataset. In Table 2 we show the compression ratio ρ . For example, for $\epsilon = 10^{-2}$ SZ3.1 achieves $\rho \approx 199$ meaning that it compresses the 1.7 TB dataset down to 8.6 GB. SZ3.1 achieves consistently higher ρ than all ML-based predictors for all error bounds, whereas CRA5 is the best among the ML-based methods. As the error bound tightens, all methods converge since the tolerance leaves little prediction-dependent room.

Recall from Section 2 that the formula for ρ considers the compressed data as well as all overhead; for ML-based methods the model weights are included in the overhead. We claim that in many cases this is unfair because the ML model is constant; consequently, its cost is amortized among numerous compression tasks. For example, we use only a small extract of the ERA5 dataset, whose total size in ARCO Zarr format is around 5 PB and continues to grow. We can use the same ML models to compress all these data and all future updates. For these reasons, we define compression ratio $\bar{\rho}$ that excludes the overhead of the ML model weights.

Table 2: Compression ratio for the entire dataset for relative error bound $\epsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. We report compression ratio ρ that includes all overhead, as well as $\bar{\rho}$ that excludes the size of the ML-model weights.

Method	$\epsilon = 10^{-2}$		$\epsilon = 10^{-3}$		$\epsilon = 10^{-4}$	
	ρ	$\bar{\rho}$	ρ	$\bar{\rho}$	ρ	$\bar{\rho}$
SZ3.1	198.80	–	36.07	–	11.72	–
CRA5	130.54	136.68	19.83	19.96	7.05	7.07
GraphCast	59.87	60.15	13.77	13.78	5.35	5.35
Aurora	55.87	109.72	14.35	16.41	5.20	5.45

Table 2 also shows $\bar{\rho}$, where it is clear that the compression ratio of all ML-based methods is improved. This is particularly evident for Aurora; for instance, for $\epsilon = 10^{-2}$, $\rho \approx 56$ whereas $\bar{\rho} \approx 110$. This happens because its residuals compress so well that the size of the model weights rivals the entire compressed dataset, roughly halving its compression ratio. CRA5 and GraphCast are less affected, and the effect fades at tighter bounds as the growing residuals dominate the fixed cost. Obviously, SZ3.1 is not affected because it does not store model weights. For the rest of the paper we will report compression ratio using $\bar{\rho}$.

In summary, none of the ML-based predictors is better than SZ3.1. However this statement does not account for the fact that individual variables behave differently, and does not consider the fidelity of the compressed data. We will investigate these issues below.

4.3 Compression and Decompression Speed

Although this is not the focus of this paper, for completeness we show in Table 3 the end-to-end compression and decompression throughput, measured as MB/s of uncompressed data. As expected, SZ3.1 is the fastest because of the simple polynomial predictor. CRA5 achieves roughly half of SZ3.1’s throughput because the calculation of the forward pass of its autoencoder is more complex. Observe that both auto-regressive methods, GraphCast and Aurora are at least one order of magnitude slower. For Aurora this is expected due to the large model size. GraphCast however, is slow because it is executed on a CPU. Recall from Section 4.1.2 that we could not employ a GPU due to the non-deterministic implementation. It must be noted that we did not attempt to optimize any method with regard to execution speed. There are many opportunities for memory and I/O optimization as well as parallelization, but they fall outside the scope of this paper.

4.4 Reconstruction Quality

Although all methods guarantee the relative error bound ϵ , they generate different error distributions that affect the quality of the decompressed data, as shown below.

4.4.1 Pointwise Error Statistics. We first ask how much the methods differ in *average* error under a common bound. Table 4 reports per-variable MAE and RMSE at $\epsilon = 10^{-2}$. Aurora improves on the smoothest fields (mean sea level pressure and geopotential), reducing MAE by 27–34% over SZ3.1 by concentrating residuals

Table 3: Throughput (MB/s of uncompressed data) at $\epsilon = 10^{-2}$, averaged over five timesteps. Rates partly reflect the per-method hardware (Section 4.1.2).

Method	Compress (MB/s)	Decompress (MB/s)
SZ3.1	348.9	1384.8
CRA5	237.8	723.2
GraphCast	30.7	33.3
Aurora	36.1	46.0

in the zero bin. However, in terms of the winds and specific humidity it is not better than SZ3.1. CRA5 goes further, producing the lowest MAE on 8 out of 9 variables, including a 91% reduction on geopotential, with the exception of specific humidity, which is less structured. The gain is bound-dependent, however: at $\epsilon = 10^{-3}$ SZ3.1 has lowest MAE for most variables. (refer to technical report⁶)

4.4.2 Error Distribution Analysis. Figure 6 shows the CDF of pointwise absolute error over all timesteps. At $\epsilon = 10^{-2}$ the ordering is variable-dependent and tracks the fidelity results: Aurora rises fastest for the smooth fields (MSL, geopotential) but not for temperature or humidity, while CRA5 leads on all but specific humidity. At $\epsilon = 10^{-3}$ the picture reverses: SZ3.1 now rises fastest for every variable, with only CRA5 still ahead on the smooth fields. This crossover is a central finding: temporal prediction sharpens the error distribution at moderate tolerances but not at tight ones, where narrow quantization bins let SZ3.1’s spatially coherent errors quantize more efficiently than the scattered errors of the temporal models. The practical consequence is application-dependent: learned prediction gives better aggregate fidelity for predictable fields at moderate bounds, whereas at tight bounds SZ3.1 wins on both compression ratio and average error.

4.4.3 Spatial Error Structure. Figure 7 maps the time-averaged per-pixel MAE at $\epsilon = 10^{-2}$. The temporal predictors err in step with local predictability: for temperature the error concentrates along the mid-latitude storm tracks (40–60° N/S) while the tropics stay near-zero, and Aurora’s MSL error is low across the tropics but rises at the high latitudes of deep cyclones. SZ3.1 is qualitatively different: its error is spatially uniform and uncorrelated with any weather feature. CRA5 differs from both: its error follows sharp geographic gradients (coastlines, land-sea contrasts) rather than weather. SZ3.1 gives better pointwise fidelity but introduces local artifacts, especially in T and MSL, so it is not always spatially smooth. While forecast-model reconstructions give worse pointwise accuracy but smoother errors and more meteorologically coherent.

4.5 In-Depth Analysis: Temporal Predictors

Temporal prediction does not, on its own, outperform *spatial* polynomial prediction for overall compression. Below we analyze the per-variable and per-level compression ratio, initialization overhead, and stability with SZ3.1 as the baseline. CRA5 is excluded: reconstructing each timestep independently, it has no overhead, no feedback, and no per-variable or per-level residual breakdown.

Table 4: Mean absolute error (MAE) and root mean square error (RMSE) per variable at $\epsilon = 10^{-2}$, averaged over all 1,997 timesteps (atmospheric variables averaged across Aurora’s 13 levels). Bold marks the lowest error per row.

Var	SZ3.1		CRA5		GraphCast		Aurora	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
T2m	4.3e-01	5.3e-01	3.6e-01	4.8e-01	4.2e-01	5.3e-01	4.6e-01	5.6e-01
U10	2.5e-01	3.0e-01	2.1e-01	2.8e-01	3.1e-01	3.7e-01	2.9e-01	3.5e-01
V10	2.4e-01	3.0e-01	2.0e-01	2.6e-01	3.0e-01	3.6e-01	2.9e-01	3.5e-01
MSL	4.4e+01	5.4e+01	7.2e+00	1.2e+01	5.9e+01	7.0e+01	3.2e+01	4.1e+01
T	2.4e-01	3.0e-01	1.4e-01	1.9e-01	2.9e-01	3.5e-01	2.8e-01	3.4e-01
U	4.5e-01	5.5e-01	3.6e-01	4.7e-01	6.1e-01	7.2e-01	5.8e-01	7.0e-01
V	4.6e-01	5.5e-01	3.2e-01	4.3e-01	6.1e-01	7.3e-01	5.9e-01	7.1e-01
Q	3.6e-05	4.4e-05	4.0e-05	4.9e-05	4.3e-05	5.1e-05	3.8e-05	4.8e-05
Z	5.6e+01	6.8e+01	5.2e+00	7.4e+00	8.3e+01	9.8e+01	3.7e+01	4.7e+01

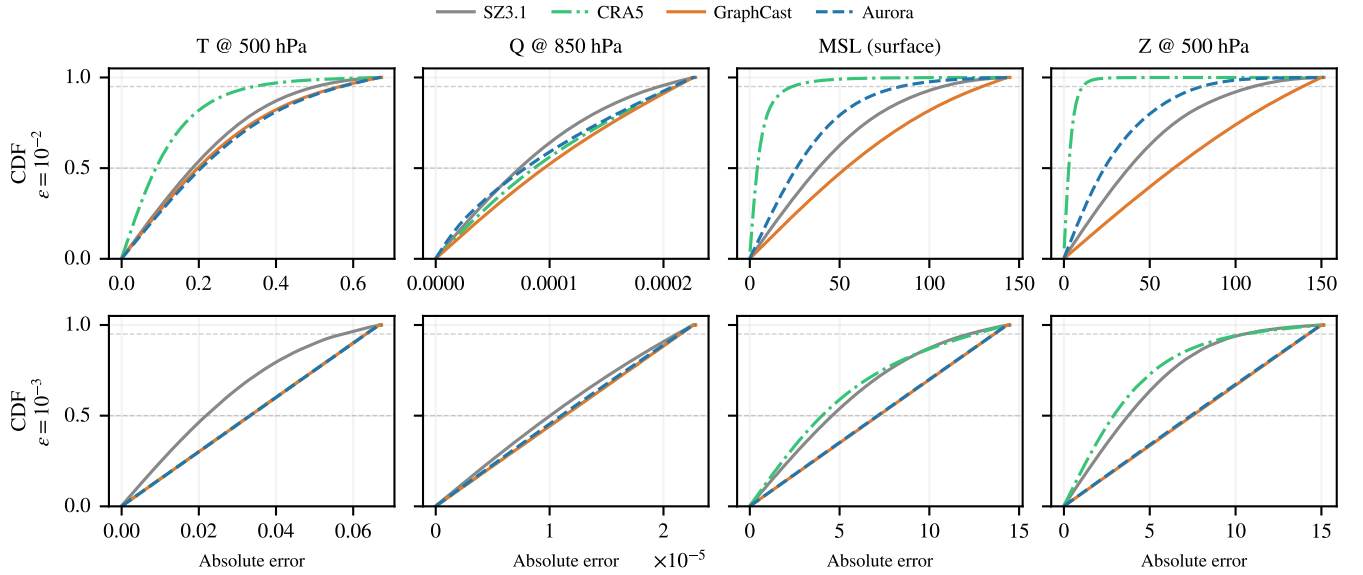


Figure 6: CDFs of pointwise absolute error at $\epsilon = 10^{-2}$ (top) and $\epsilon = 10^{-3}$ (bottom) for four representative variables; a steeper rise means error concentrated nearer zero. Dashed lines mark the median and 95th percentile.

4.5.1 Per-Variable Compression Ratio. In Figure 8 we present the per-variable compression ratio, excluding the initialization overhead. At $\epsilon = 10^{-2}$ Aurora wins on the smoothest fields: geopotential compresses about 8,540 $\times$ against SZ3.1’s 888 $\times$, with comparable gains for mean sea level pressure and 2-meter temperature, because near-zero residuals leave quantized streams dominated by zeros. On the more turbulent fields, atmospheric temperature, the winds, and specific humidity, SZ3.1 keeps the advantage, as their fine-scale variability resists grid-scale prediction. GraphCast trails both on every field except 2-meter temperature, likely because its mesh-to-grid interpolation injects high-frequency artifacts. At $\epsilon = 10^{-3}$ the advantage disappears and SZ3.1 leads every variable. As established in Section 4.4, the benefit of small residuals is confined to moderate tolerances, since tight bins span a wide integer range regardless of residual magnitude.

4.5.2 Per-Level Compression Ratio. We ask how this varies with altitude (Figure 9). For temperature, Aurora is best in the smooth mid-troposphere, peaking near 580 $\times$ at 600 hPa, but is overtaken by SZ3.1 above 300 hPa and matched by it below 850 hPa. Humidity compresses well for every method at the near-dry upper levels (50–100 hPa) and drops sharply below 300 hPa, where SZ3.1 leads.

Geopotential is the most extreme: Aurora climbs from about 670 $\times$ near the surface to over 45,000 $\times$ at the top of the atmosphere, where its predictions are nearly perfect. These per-level peaks do not survive aggregation, however: the variable total is dominated by the lower-tropospheric levels, which carry the largest compressed sizes and thus most of the bytes (a harmonic-mean effect).

4.5.3 Initialization Overhead and Amortization. The temporal methods store two uncompressed seed timesteps, so we check whether

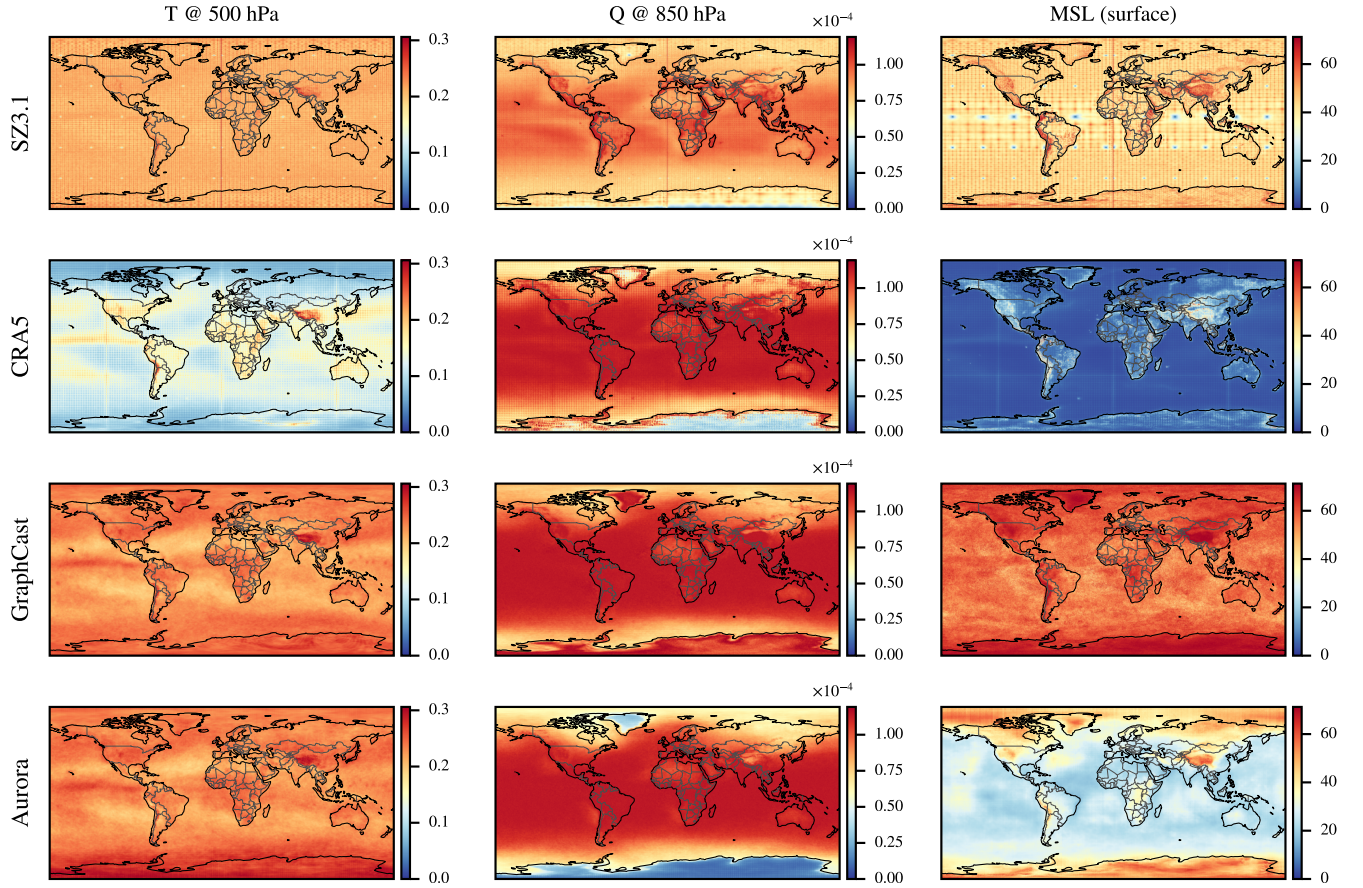


Figure 7: Time-averaged per-pixel MAE at $\epsilon = 10^{-2}$. Rows: Aurora, GraphCast, SZ3.1, CRA5. Columns: temperature (500 hPa), specific humidity (850 hPa), mean sea level pressure. Color scale shared within each column.

this fixed overhead materially lowers their ratios (Figure 10). Convergence to the asymptotic ratio depends sharply on the bound. When residuals are large ($\epsilon = 10^{-4}$), the overhead becomes negligible within roughly 80 timesteps. At $\epsilon = 10^{-2}$, by contrast, the residuals compress so well that the two seed frames still dominate the stored size after the full 1,997-step sequence.

Over the multi-year archives these methods target, the overhead amortizes at every bound; within our window it depresses the dataset-level ratio only at $\epsilon = 10^{-2}$. We therefore report dataset-level ratios (Table 2) with the overhead included and the per-variable and per-level figures (Figures 8, 9) without it, isolating intrinsic residual compressibility.

4.5.4 Autoregressive Stability. A natural concern is whether quantization artifacts compound through the feedback loop: in free-running forecasting, these models diverge after roughly 10 days (~ 40 steps). Figure 11 tracks the normalized MAE over the full 1,997-step sequence (~ 500 days) at $\epsilon = 10^{-2}$: all three methods stay stationary around 1.0 with no upward trend. The closed loop guarantees this, feeding each prediction from *decompressed* states within δ_v of the truth so the input cannot drift far from reality. The $\pm 5\%$ oscillations reflect seasonal variation in predictability, not

drift; SZ3.1’s are marginally larger because its error tracks each timestep’s spatial complexity, whereas the temporal predictors’ curves are smoother. The rollout is therefore stable over the entire sequence, so the error-bounded feedback removes the drift that limits free-running forecasts.

4.6 Residual-Structure Analysis

The learned predictors often reconstruct better than SZ3.1, yet do not yield a higher compression ratio. To explain this, note that with the shared back-end the ratio is fixed by the quantized-residual stream, not by reconstruction accuracy: at a fixed error bound, $CR = \frac{B_0}{R}$ and $R \approx H(Q)$, where B_0 is the raw bit-width per value, R the achieved bits per value, and $H(Q)$ the symbol entropy of the quantized residual, the floor the Huffman stage works against. Compressibility is therefore governed by residual *entropy*, and a more accurate prediction need not yield a lower-entropy residual.

Table 5 separates two error measures that are usually conflated, for two representative fields (a smooth surface field and a turbulent mid-tropospheric one). MAE_{rec} is the *reconstruction* error [truth – recon], the fidelity the user observes, and averages over *all* points, so it is dominated by the well-predicted majority. MAE_{res} is the

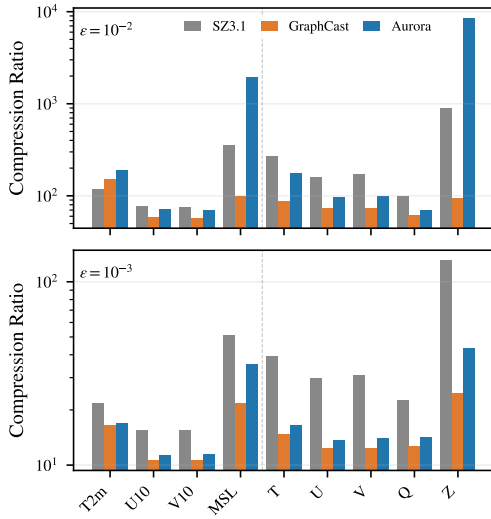


Figure 8: Per-variable compression ratio at $\epsilon = 10^{-2}$ (top) and $\epsilon = 10^{-3}$ (bottom), excluding initialization overhead; log-scale y-axis. GraphCast and SZ3.1 use Aurora’s 13 levels.

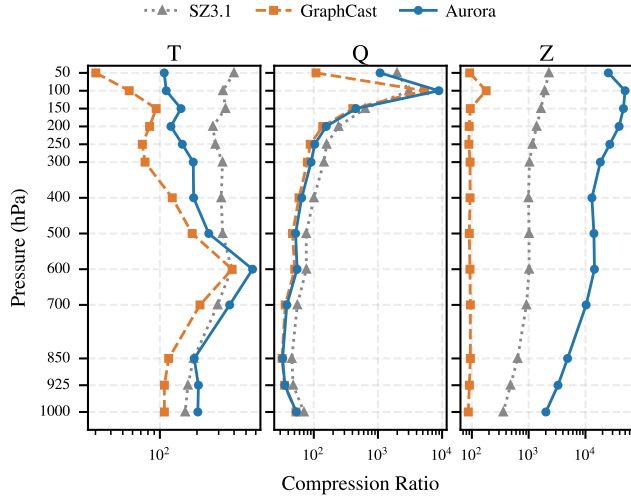


Figure 9: Per-level compression ratio for temperature, specific humidity, and geopotential at $\epsilon = 10^{-2}$ (Aurora’s 13 levels), excluding initialization overhead.

residual magnitude $|2q\epsilon|$, the mean size of the corrections actually stored; it depends on how *many* points exceed the bound (*zero %* counts those within it) and how large their corrections are. MAE_{res} , not fidelity, tracks $H(Q)$ and hence the rate. The two measures can move in *opposite* directions: a predictor can be excellent on the bulk (low MAE_{rec}) yet correct enough outliers to inflate the stored residual, which is precisely what costs bits.

Better fidelity need not lower the entropy. On T2m, CRA5 has the *best* reconstruction MAE (0.36 vs. SZ3.1’s 0.43) but the *largest* residual MAE (0.21 vs. 0.12): it predicts most points accurately yet leaves

Table 5: Reconstruction error, residual magnitude, zero-bin fraction, entropy $H(Q)$, and compression ratio for two representative fields ($\epsilon = 10^{-2}$, 200 timesteps). Bold marks cases discussed in the text.

Var	Method	MAE_{rec}	MAE_{res}	zero %	$H(Q)$	CR
T2m	SZ3.1	4.3e-01	1.2e-01	96.2	0.3	117.4
	CRA5	3.6e-01	2.1e-01	92.4	0.5	103.5
	GraphCast	4.3e-01	1.4e-01	94.6	0.4	145.9
	Aurora	4.6e-01	1.2e-01	95.4	0.3	177.8
Q@850	SZ3.1	8.3e-05	7.9e-05	88.6	0.8	47.9
	CRA5	9.4e-05	1.4e-04	74.2	1.3	34.0
	GraphCast	1.0e-04	1.7e-04	71.2	1.5	33.4
	Aurora	8.9e-05	1.9e-04	70.5	1.5	34.0

a correction at more of them (7.6% outside the bound vs. 3.8%), so its residual entropy is the highest (0.5 vs. 0.3 bits) and its CR the lowest (103.5 vs. 117.4). On 850 hPa specific humidity, Aurora matches SZ3.1’s fidelity (8.9 vs. 8.3×10^{-5}) yet stores a residual 2.4× larger (1.9 vs. 0.8×10^{-4}), with entropy nearly double (1.5 vs. 0.8 bits), corresponding to corrections at 30% of points vs. 11%, so its CR is 34.0 vs. 47.9. Equal-or-better fidelity yields a larger stored residual and lower compression.

Consequence for the dataset. The learned methods’ fidelity advantage therefore does not become a dataset-level CR gain. The aggregate ratio is byte-weighted toward the turbulent, high-entropy fields (humidity, winds), where their residuals carry up to twice SZ3.1’s entropy, while the smooth fields on which they win (geopotential, MSL) contribute little. The advantage is also mainly CRA5’s: Aurora and GraphCast do not perform better than SZ3.1 on the mid-tropospheric and turbulent fields.

Where the corrections occur. Figure 12 maps the points at which each method stores a correction. SZ3.1’s form a spatially *unstructured* scatter of isolated pixels, whereas every learned method concentrates its corrections into coherent regions following coastlines and weather features. This matters because the back-end is order-0 (Huffman prices each symbol by its global frequency) followed by a one-dimensional run/dictionary stage (Zstandard, which exploits only exact repeats along the row-major scan), so the *two-dimensional* structure of the learned residuals is largely invisible to it. A predictor whose errors are spatially organized leaves compression unrealized under a conventional 1-D pipeline, which points to spatially-aware (2-D) residual coding as a route to convert the learned methods’ prediction quality into compression ratio.

5 Related Work

5.1 Classical Error-Bounded Lossy Compression

Among error-bounded compressors, ZFP [29] operates in fixed-rate mode by truncating coefficients after an orthogonal block transform, while SPERR [25] applies the SPECK wavelet algorithm [35, 38] to structured scientific grids. The SZ family represents the most widely adopted class of such compressors. SZ2+ [39] introduced multidimensional Lorenzo predictors; SZ3 [28] and SZauto [43]

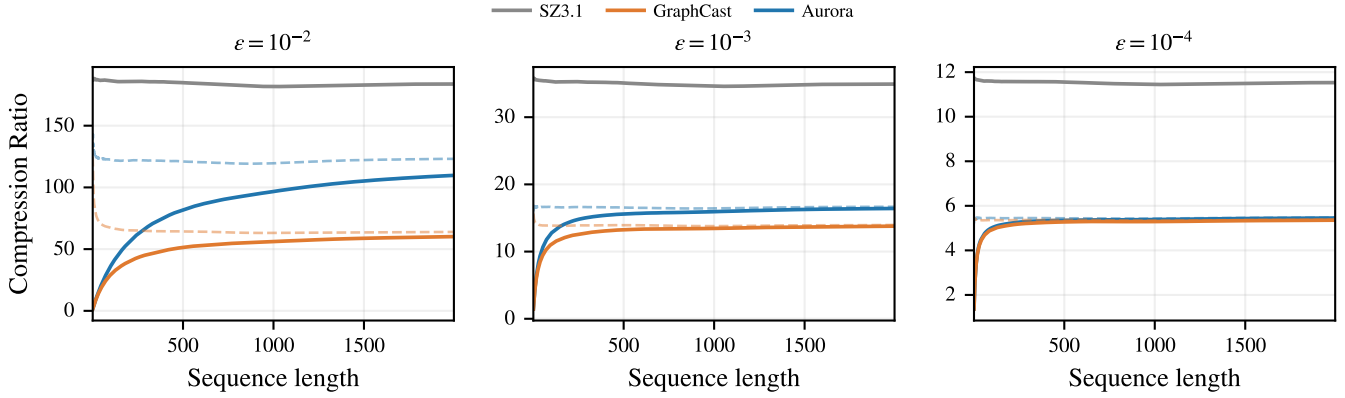


Figure 10: Compression ratio versus sequence length for all three methods and error bounds. Solid lines include the two-timestep initialization overhead in the denominator; dashed lines give the asymptotic CR without it (SZ3.1 has none).

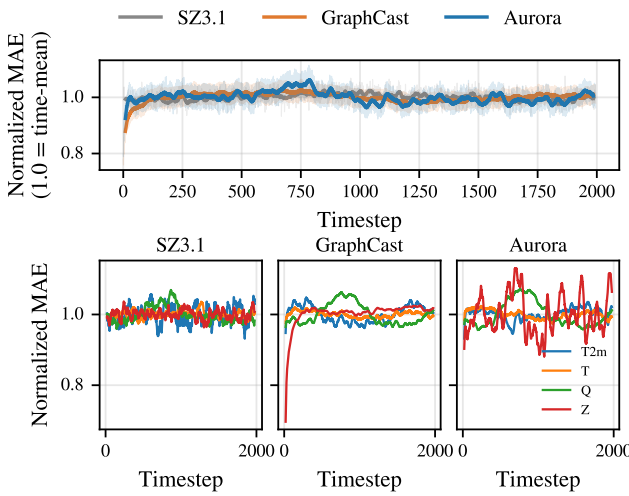


Figure 11: Normalized MAE over the 1,997-step autoregressive rollout at $\epsilon = 10^{-2}$; each variable’s MAE is divided by its time-mean. Top: per-method average over all variables (bold: 20-step rolling average; shaded: raw values). Bottom: per-variable curves for each method (representative variables).

extended the prediction and quantization pipeline; and SZ3.1 [41] incorporated dynamic multidimensional spline interpolation. Recently, HPEZ [32] added natural cubic splines, multidimensional interpolation strategies, and quality-driven auto-tuning. All of these methods predict from spatially neighboring points within a single timestep, leaving temporal redundancy not entirely exploited. We evaluate against SZ3.1 [41] as the sole classical baseline. Although HPEZ [32] is more recent and extends SZ3.1 with additional interpolation strategies, our preliminary experiments on a small subset of the data showed that SZ3.1 matches or closely approaches HPEZ in compression ratio across several tested variables and error bounds, with any gains from HPEZ remaining marginal.

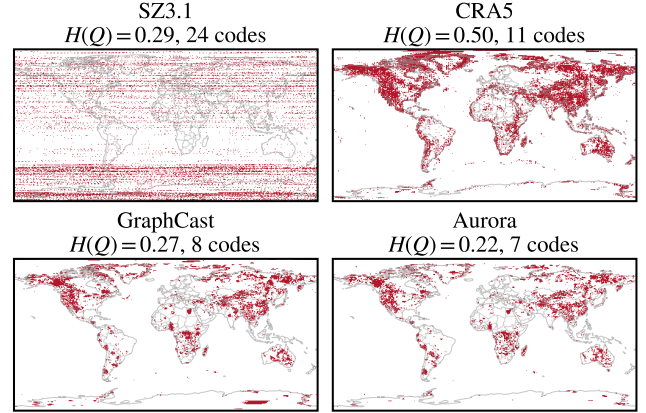


Figure 12: Binary quantized-residual support for T2m at one timestep ($\epsilon = 10^{-2}$): red marks where a correction is stored, white where the prediction is within the bound.

5.2 ML-based Lossy Compression

INR: Implicit Neural Representations-based Compression. CoordNet [10] represents time-varying volumetric fields as a coordinate-based MLP mapping (x, y, z, t) to scalar values; the network weights serve as the compressed representation, enabling continuous queries and high compression ratios. KD-INR[11] addresses the scalability problem of single-model INRs over long sequences by training compact per-timestep teacher INRs and distilling them into a single student network with temporal embeddings, reducing cross-time redundancy without sacrificing per-timestep detail. COIN++ [8] meta-learns a shared INR backbone and compresses new instances by optimizing only small FiLM-style modulations per patch, drastically reducing encoding cost while maintaining quality across diverse modalities including climate fields. Huang et al. [15] take a similar approach specifically for weather and climate data, overfitting coordinate networks to ERA5 fields and storing quantized, entropy-coded weights as the compressed representation.

Autoencoder-based Compression. HAE [26] proposes a hierarchical autoencoder in which a coarse-level AE captures global structure, and successive finer-level AEs encode residuals at increasing spatial detail; each level’s latents are independently quantized and entropy-coded, allowing rate-distortion tradeoffs.

Hayne et al. [13] applied a variational autoencoder [2] to 2D floating-point scientific fields, achieving high compression ratios without error guarantees.

Hybrid methods with error bounds. AE-SZ [31] integrates a learned convolutional AE predictor with the classical Lorenzo predictor inside the SZ framework. SRN-SZ [30] reframes compression as a downsample-then-super-resolve pipeline: the encoder stores a coarsened grid, and a high-accuracy super-resolution network (HAT) reconstructs fine spatial detail at decode time, falling back to adaptive interpolation when the resolution gap is too large. GraphComp [23] maps each timestep to a graph of segmented regions and trains a temporal graph autoencoder whose latent compresses the sequence.

LLMComp [24] casts compression as auto-regressive token modeling, training a decoder-only transformer that stores a top- k rank index when it predicts the next token and a fallback correction otherwise. These hybrid approaches demonstrate that learned predictors can improve compression ratio at fixed error bounds, but all remain domain-agnostic and some do not explicitly exploit the specific spatiotemporal structure of atmospheric data.

We do not evaluate any of the mentioned autoencoder-based or hybrid methods because: (i) they require training from scratch on the ERA5 data, which is a substantial undertaking that defies the purpose of this work; (ii) for AE-SZ and SRN-SZ the code is not publicly available. We also note that HPEZ and SZ3.1 represent the state-of-the-art amongst all SZ-based works. Therefore, evaluating against SZ3.1 provides a strong baseline for comparison.

5.3 Neural Compression for ERA5

We have already discussed CRA5 [12] in the previous sections.

Another neural compressor for ERA5 proposed by Huang et al. [15] takes a hierarchical neural approach targeting the full ERA5 variable set with similar goals. Both are purpose-built ERA5 codecs with strong results, but their standard evaluation treats the full codec as a monolithic system operating outside an error-bounded framework. Huang et al. [15] is not evaluated here because there is no publicly available pre-trained model and training from scratch is required. Therefore, its integration is left for future work.

5.4 Weather Forecasting ML Models

Beyond GraphCast [21] and Aurora [4] that have been discussed before, a broader landscape of data-driven forecasters has been proposed. Pangu-Weather [3] uses a 3D Earth-Specific Transformer with latitude-dependent positional biases and separate model instances for different lead times to limit auto-regressive error accumulation. FourCastNet [34] applies Adaptive Fourier Neural Operators that learn spectral filters in the frequency domain, enabling global receptive fields with favorable computational scaling. FengWu[5] extends this family with multi-lead supervision and uncertainty-aware losses targeting skill horizons beyond 10 days. GenCast [36] introduces a diffusion-based probabilistic forecaster

on a spherical mesh, generating calibrated forecast ensembles that improve representation of extreme events. Aardvark [40] replaces the full NWP inference stack with a trainable encoder that ingests raw observations, enabling end-to-end optimization without NWP inputs at deployment time.

These models are excluded from our evaluation due to limited variable coverage, less accessible inference pipelines, or architectural choices that complicate integration into a deterministic prediction-residual framework (e.g., GenCast’s probabilistic diffusion outputs). We leave their integration for future work.

6 Limitations and Future Directions

Our study identifies three main limitations of using ML-based predictors for bounded lossy compression: (i) *Computational and storage cost.* Running a deep neural network in place of a polynomial predictor is computationally costly; the pretrained weights are also a large fixed side cost stored wherever data is decoded. (ii) *Sequential decoding.* For auto-regressive methods, reconstructing timestep k requires rolling the model forward from the seed states through every step in between. Note that CRA5 does not suffer from this problem. (iii) *Compression ratio.* In general, classical compressors achieve higher compression ratio, although ML-based methods can be better for specific variables and may achieve higher fidelity.

There are several opportunities for extending this work: (i) *Additional ML-models.* Extend the evaluation to include more forecasting models, such as Pangu-Weather, FourCastNet and FengWu. (ii) *Structure-aware residual coding.* Encode the residual to exploit its spatial organization through spatially adaptive quantization.

(iii) *Compression-aware predictors.* Train predictors with a residual-entropy objective, instead of mean squared error, to match the requirements of the Huffman encoder.

7 Conclusion

In this Experimental Analysis and Benchmark (EA&B) paper, we present the first systematic evaluation of deep learning weather models as predictors in an error-bounded compression pipeline for ERA5: a learned spatial codec (CRA5) and two temporal forecasters Aurora, GraphCast against the SZ3.1 baseline. We propose an auto-regressive framework that bounds error at every step and avoids error accumulation. The headline result is negative: no learned predictor improves dataset-level compression, and SZ3.1 leads at every bound. Yet the ML-based predictors may offer higher fidelity and higher compression ratio for smooth variables. We analyze why higher fidelity does not translate to better compression: the ratio is determined by residual entropy, not accuracy, and the learned residuals are spatially structured in a way the one-dimensional coder cannot exploit. We argue that our study is a sound proxy that exposes the bottlenecks of using learned predictors for the general case of scientific data compression.

Acknowledgments

For computer time, we used IBEX and Shaheen III, managed by the Supercomputing Core Laboratory at KAUST, Saudi Arabia.

References

- [1] Miranda application. [n. d.]. *Miranda*. <https://wci.llnl.gov/simulation/computer-codes/miranda>
- [2] Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. 2018. Variational image compression with a scale hyperprior. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [3] Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. 2023. Accurate medium-range global weather forecasting with 3D neural networks. *Nature* 619, 7970 (2023), 533–538.
- [4] Cristian Bodnar, Wessel Bruinsma, Ana Lucic, Megan Stanley, Johannes Vaughan, Leonardo Brandão, Megan Garvan, Max Riechert, Jonathan Weyn, Haiyu Dong, et al. 2024. Aurora: A foundation model of the atmosphere. *arXiv preprint arXiv:2405.13063* (2024).
- [5] Kang Chen, Tao Han, Junchao Gong, Lei Bai, Fenghua Ling, Jing-Jia Luo, Xi Chen, Leiming Ma, Tianning Zhang, Rui Su, et al. 2023. Fengwu: Pushing the skillful global medium-range weather forecast beyond 10 days lead. *arXiv preprint arXiv:2304.02948* (2023).
- [6] Wai Tong Chung, Ki Sung Jung, Jacqueline H. Chen, and Matthias Ihme. 2022. BLASTNet: A call for community-involved big data in combustion machine learning. *Applications in Energy and Combustion Science* 12 (2022), 100087. doi:10.1016/j.jaecs.2022.100087
- [7] Hari Prasad Dasari, Yesubabu Viswanadhapalli, Sabique Langodan, Yasser Abualnaja, Srinivas Desamsetti, Koteswararao Vankayalapati, Luong Thang, and Ibrahim Hoteit. 2022. High-resolution climate characteristics of the Arabian Gulf based on a validated regional reanalysis. *Meteorological Applications* 29, 5 (2022), e2102.
- [8] Emilien Dupont, Hrshikesh Loya, Milad Alizadeh, Adam Golinski, Yee Whye Teh, and Arnaud Doucet. 2022. Coin++: neural compression across modalities. *Transactions on Machine Learning Research* (2022).
- [9] Jim Gray, David T Liu, Maria Nieto-Santesteban, Alex Szalay, David J DeWitt, and Gerd Heber. 2005. Scientific data management in the coming decade. *ACM SIGMOD Record* 34, 4 (2005), 34–41.
- [10] Jun Han and Chaoli Wang. 2022. Coordnet: data generation and visualization generation for time-varying volumes via a coordinate-based neural network. *IEEE Transactions on Visualization and Computer Graphics* (2022).
- [11] Jun Han, Hao Zheng, and Chongke Bi. 2023. Kd-inr: Time-varying volumetric data compression via knowledge distillation-based implicit neural representation. *IEEE Transactions on Visualization and Computer Graphics* 30, 10 (2023), 6826–6838.
- [12] Tao Han, Zhenghao Chen, Song Guo, Wanghan Xu, and Lei Bai. 2024. CRA5: Extreme compression of ERA5 for portable global climate and weather research via an efficient variational transformer. *arXiv preprint arXiv:2405.03376* (2024).
- [13] Lucas Hayne, John Clyne, and Shaomeng Li. 2021. Using neural networks for two dimensional scientific data compression. In *Proceedings of the IEEE international conference on Big Data (Big Data)*. 2956–2965.
- [14] Hans Hersbach, Bill Bell, Paul Berrisford, Shoji Hirahara, András Horányi, Joaquín Muñoz-Sabater, Julien Nicolas, Carole Peubey, Raluca Radu, Dinand Schepers, et al. 2020. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society* 146, 730 (2020), 1999–2049.
- [15] Langwen Huang and Torsten Hoefler. 2023. Compressing multidimensional weather and climate data into neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [16] David A. Huffman. 1952. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101. doi:10.1109/JRPROC.1952.273898
- [17] Ian T Jolliffe and Jorge Cadima. 2016. Principal component analysis: a review and recent developments. *Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences* 374, 2065 (2016), 20150202.
- [18] JE Kay, C Deser, A Phillips, A Mai, C Hannay, G Strand, JM Arblaster, S Bates, G Danabasoglu, J Edwards, et al. 2015. The Community Earth System Model (CESM) large ensemble project: A community resource for studying climate change in the presence of internal climate variability. *Bulletin of the American Meteorological Society* 96, 8 (2015), 1333–1349.
- [19] Suha N Kayum, Thierry Tonellot, Vincent Etienne, Ali Momin, and et al. 2020. GeoDRIVE-a high performance computing flexible platform for seismic applications. *First Break* 38, 2 (2020), 97–100.
- [20] Virginia Klema and Alan Laub. 1980. The singular value decomposition: its computation and some applications. *IEEE Transactions on automatic control* 25, 2 (1980), 164–176.
- [21] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, et al. 2023. Learning skillful medium-range global weather forecasting. *Science* 382, 6677 (2023), 1416–1421.
- [22] Sabique Langodan, Luigi Cavaleri, Angela Pomaro, Jesus Portilla, Yasser Abualnaja, and Ibrahim Hoteit. 2018. Unraveling climatic wind and wave trends in the red sea using wave spectra partitioning. *Journal of Climate* 31, 5 (2018), 1881–1895. doi:10.1175/JCLI-D-17-0295.1
- [23] Guozhong Li, Muhannad Alhumaidi, Spiros Skiadopoulos, Ibrahim Hoteit, and Panos Kalnis. 2026. Extreme Error-bounded Compression of Scientific Data via Temporal Graph Autoencoders. *IEEE Transactions on Knowledge and Data Engineering* (2026).
- [24] Guozhong Li, Muhannad Alhumaidi, Spiros Skiadopoulos, and Panos Kalnis. 2025. LLMComp: A Language Modeling Paradigm for Error-Bounded Scientific Data Compression. In *Proceedings of the IEEE international conference on Big Data (Big Data)*.
- [25] Shaomeng Li, Peter Lindstrom, and John Clyne. 2023. Lossy scientific data compression with SPERR. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 1007–1017.
- [26] Xiao Li, Jaemoon Lee, Anand Rangarajan, and Sanjay Ranka. 2024. Attention based machine learning methods for data reduction with guaranteed error bounds. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 1039–1048.
- [27] Yi Li, Eric Perlman, Minping Wan, Yunke Yang, Charles Meneveau, Randal Burns, Shiyi Chen, Alexander Szalay, and Gregory Eyink. 2008. A public turbulence database cluster and applications to study lagrangian evolution of velocity increments in turbulence. *Journal of Turbulence* 9 (2008), N31.
- [28] Xin Liang, Kai Zhao, Sheng Di, Sihuan Li, Robert Underwood, Ali M Gok, Jiannan Tian, Jieyang Deng, Jon C Calhoun, Dingwen Tao, et al. 2022. SZ3: A modular framework for composing prediction-based error-bounded lossy compressors. *IEEE Transactions on Big Data* 9, 2 (2022), 485–498.
- [29] Peter Lindstrom. 2014. Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics* 20, 12 (2014), 2674–2683.
- [30] Jinyang Liu, Sheng Di, Sian Jin, Kai Zhao, Xin Liang, Zizhong Chen, and Franck Cappello. 2023. SRN-SZ: scientific error-bounded lossy compression with super-resolution neural networks. In *Proceedings of the IEEE international conference on Big Data (Big Data)*. 229–236.
- [31] Jinyang Liu, Sheng Di, Kai Zhao, Sian Jin, Dingwen Tao, Xin Liang, Zizhong Chen, and Franck Cappello. 2021. Exploring autoencoder-based error-bounded compression for scientific data. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 294–306.
- [32] Jinyang Liu, Sheng Di, Kai Zhao, Xin Liang, Sian Jin, Zizhe Jian, Jiajun Huang, Shixun Wu, Zizhong Chen, and Franck Cappello. 2024. High-performance effective scientific error-bounded lossy compression with auto-tuned multi-component interpolation. *SIGMOD* 2, 1 (2024), 1–27.
- [33] NYX simulation. 2019. NYX. <https://amrex-astro.github.io/Nyx>
- [34] Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, et al. 2022. FourCastNet: A global data-driven high-resolution weather forecasting model using adaptive Fourier neural operators. *arXiv preprint arXiv:2202.11214* (2022).
- [35] William A Pearlman, Asad Islam, Nithin Nagaraj, and Amir Said. 2004. Efficient, low-complexity image coding with a set-partitioning embedded block coder. *IEEE transactions on circuits and systems for video technology* 14, 11 (2004), 1219–1235.
- [36] Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, Tom R Andersson, Andrew El-Kadi, Dominic Masters, Timo Ewalds, Jacklynn Stott, Shakir Mohamed, Peter Battaglia, et al. 2023. Gencast: Diffusion-based ensemble forecasting for medium-range weather. *arXiv preprint arXiv:2312.15796* (2023).
- [37] Russ Rew and Glenn Davis. 1990. NetCDF: an interface for scientific data access. *IEEE computer graphics and applications* 10, 4 (1990), 76–82.
- [38] Xiaoli Tang and William A Pearlman. 2006. Three-dimensional wavelet-based compression of hyperspectral images. In *Hyperspectral data compression*. Springer, 273–308.
- [39] Dingwen Tao, Sheng Di, Zizhong Chen, and Franck Cappello. 2017. Significantly improving lossy compression for scientific data sets based on multidimensional prediction and error-controlled quantization. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 1129–1139.
- [40] Anna Vaughan, Stratis Markou, Will Tebbutt, James Requeima, Wessel P Bruinsma, Tom R Andersson, Michael Herzog, Nicholas D Lane, Matthew Chantry, J Scott Hosking, et al. 2024. Aardvark weather: end-to-end data-driven weather forecasting. *arXiv preprint arXiv:2404.00411* (2024).
- [41] Kai Zhao, Sheng Di, Maxim Dmitriev, Thierry-Laurent D Tonellot, Zizhong Chen, and Franck Cappello. 2021. Optimizing error-bounded lossy compression for scientific data by dynamic spline interpolation. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1643–1654.
- [42] Kai Zhao, Sheng Di, Xin Lian, Sihuan Li, Dingwen Tao, Julie Bessac, Zizhong Chen, and Franck Cappello. 2020. SDRBench: scientific data reduction benchmark for lossy compressors. In *Proceedings of the IEEE international conference on Big Data (Big Data)*. 2716–2724.
- [43] Kai Zhao, Sheng Di, Xin Liang, Sihuan Li, Dingwen Tao, Zizhong Chen, and Franck Cappello. 2020. Significantly improving lossy compression for HPC datasets with second-order prediction and parameter optimization. In *Proceedings of the International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. 89–100.

Table 6: DL prediction and compression models considered in this survey, organized by scope. Selection criteria for our evaluation are recency, code availability, and native applicability to ERA5 atmospheric data.

Model	Scope	Code Available	ERA5 Native
CoordNet	Time-Varying Volumes	✓	✗
KD-INR	Time-Varying Volumes	✗	✗
HAE	General	✓	✗
ATN-AE	General	✗	✗
GraphComp	General	✓	✗
AE-SZ	General	✗	✗
SRN-SZ	General	✗	✗
COIN++	General	✓	✗
CRA5	Weather (ERA5)	✓	✓
HH-NN	Weather (ERA5)	✓	✓
GraphCast	Weather-Forecasting	✓	✓
GenCast	Weather-Forecasting	✓	✓
Aurora	Weather-Forecasting	✓	✓
Aardvark	Weather-Forecasting	✓	✓
Pangu-Weather	Weather-Forecasting	✓	✓
FengWu	Weather-Forecasting	✓	✓
FourCastNet	Weather-Forecasting	✓	✓

Table 7: Mean absolute error (MAE) and root mean square error (RMSE) per variable at $\epsilon = 10^{-3}$, averaged over all timesteps. Bold marks the lowest error per row.

Var	SZ3.1		CRA5		GraphCast		Aurora	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
T2m	5.7e-02	6.8e-02	6.5e-02	7.5e-02	6.5e-02	7.5e-02	6.5e-02	7.5e-02
U10	3.3e-02	3.9e-02	3.5e-02	4.1e-02	3.5e-02	4.1e-02	3.5e-02	4.1e-02
V10	3.2e-02	3.7e-02	3.4e-02	3.9e-02	3.4e-02	3.9e-02	3.4e-02	3.9e-02
MSL	5.2e+00	6.3e+00	4.9e+00	6.2e+00	7.2e+00	8.3e+00	7.1e+00	8.3e+00
T	3.0e-02	3.6e-02	3.8e-02	4.4e-02	3.8e-02	4.4e-02	3.8e-02	4.4e-02
U	5.7e-02	6.8e-02	7.0e-02	8.1e-02	7.0e-02	8.1e-02	7.0e-02	8.1e-02
V	5.7e-02	6.9e-02	7.0e-02	8.1e-02	7.1e-02	8.2e-02	7.1e-02	8.2e-02
Q	4.5e-06	5.4e-06	5.1e-06	5.9e-06	5.1e-06	5.9e-06	5.0e-06	5.8e-06
Z	5.4e+00	6.6e+00	4.8e+00	6.2e+00	9.3e+00	1.1e+01	9.2e+00	1.1e+01

A Appendix

A.1 Summary of Related Work

Table 6 summarizes the landscape of methods surveyed above against three selection criteria: code availability, native ERA5 applicability, and inclusion in our evaluation. We focus on pretrained DL forecasting models rather than purpose-built compression codecs for the reasons established in Section 1: training a compression-specific DL model from scratch requires substantial resources, and it is important to first establish whether DL-based temporal prediction offers a genuine advantage over classical spatial prediction before incurring that cost. Pretrained forecasting models serve as a principled proxy for this feasibility study.

the lowest MAE and RMSE on 7 out of 9 variables, including near-surface temperature, winds, atmospheric temperature, wind fields, and specific humidity. The learned residual methods provide little benefit for these less structured or higher-frequency fields, where their residuals are not concentrated enough to offset the additional correction error. CRA5 remains beneficial only for the smoothest large-scale fields, reducing MAE by about 5% on mean sea level pressure and 12% on geopotential relative to SZ3.1. In contrast, GraphCast and Aurora do not improve over SZ3.1 at this bound, and are especially worse on mean sea level pressure and geopotential. This confirms that the gains observed at $\epsilon = 10^{-2}$ are strongly bound-dependent: when the tolerance is tightened, SZ3.1 already preserves most variables accurately, leaving less exploitable residual structure for forecast-based correction.

A.2 Error Statistics for Tighter Bound

Table 7 reports per-variable MAE and RMSE at $\epsilon = 10^{-3}$. At this tighter error bound, the advantage shifts back to SZ3.1: it achieves